

## Algorithm Efficiency

### Learning Objectives

- Explain time and space complexity using Big-O notation
- Compare algorithms by analyzing their growth rates
- Apply complexity analysis to choose efficient solutions for given problems

### Engage (5-7 minutes)

Present a dramatic comparison: Algorithm A takes  $n$  steps for input size  $n$ . Algorithm B takes  $n^2$  steps. For  $n = 10$ , A takes 10 steps, B takes 100. For  $n = 1000$ , A takes 1000, B takes 1,000,000. Ask students: at what point does the difference become critical? Introduce that Big-O notation captures this scaling behavior.

### Explore (10-12 minutes)

Students receive timing data for several algorithms (linear, quadratic, logarithmic, exponential) on inputs of sizes 10, 100, 1000, and 10000. They plot the data on graphs, identify the growth patterns, and match each to its Big-O class. Discussion: why does the choice of algorithm matter more as data grows?

### Explain (10-12 minutes)

Algorithm efficiency is measured by time complexity (how runtime grows with input size) and space complexity (how memory usage grows). Big-O notation describes the upper bound of growth rate, ignoring constants and lower-order terms:  $O(1)$  constant,  $O(\log n)$  logarithmic,  $O(n)$  linear,  $O(n \log n)$  linearithmic,  $O(n^2)$  quadratic,  $O(2^n)$  exponential. For large inputs, a quadratic algorithm may be millions of times slower than a logarithmic one.

### Elaborate (8-10 minutes)

Analyze the complexity of common operations: array access  $O(1)$ , linked list search  $O(n)$ , binary search  $O(\log n)$ , matrix multiplication  $O(n^3)$ . Discuss amortized analysis—how occasional expensive operations average out over many calls. Examine practical limits: an  $O(2^n)$  algorithm is infeasible for  $n > 40$ , while  $O(n^2)$  is acceptable for  $n < 10,000$ . Introduce space-time tradeoffs explicitly.

### Evaluate (5-8 minutes)

Students analyze given algorithms and derive their Big-O complexity. Case study: a startup processes 1 million records—compare  $O(n)$ ,  $O(n \log n)$ , and  $O(n^2)$  solutions with concrete timing estimates. Self-assessment: students create a cheat sheet of common complexity classes with real-world examples.